

LLD Practice Platform — Architecture & Design Note

System architecture, clean domain model, dual-layer evaluator strategy, and service lifecycle specifications.

 Generated: September 2026  Project: LLD Practice Platform  Status: Verified & Production Ready

DESIGN.md — LLD Practice Platform Architecture & Domain Design

1. System Overview & Architecture

The **LLD Practice Platform** is built as a clean, modular, and testable monolithic web application. It bridges the gap between passive LLD study and active engineering interview practice by combining structured object-oriented design authoring with explainable deterministic and AI-powered feedback.



Syntax error in text
mermaid version 10.9.8

2. The Core Learner Journey

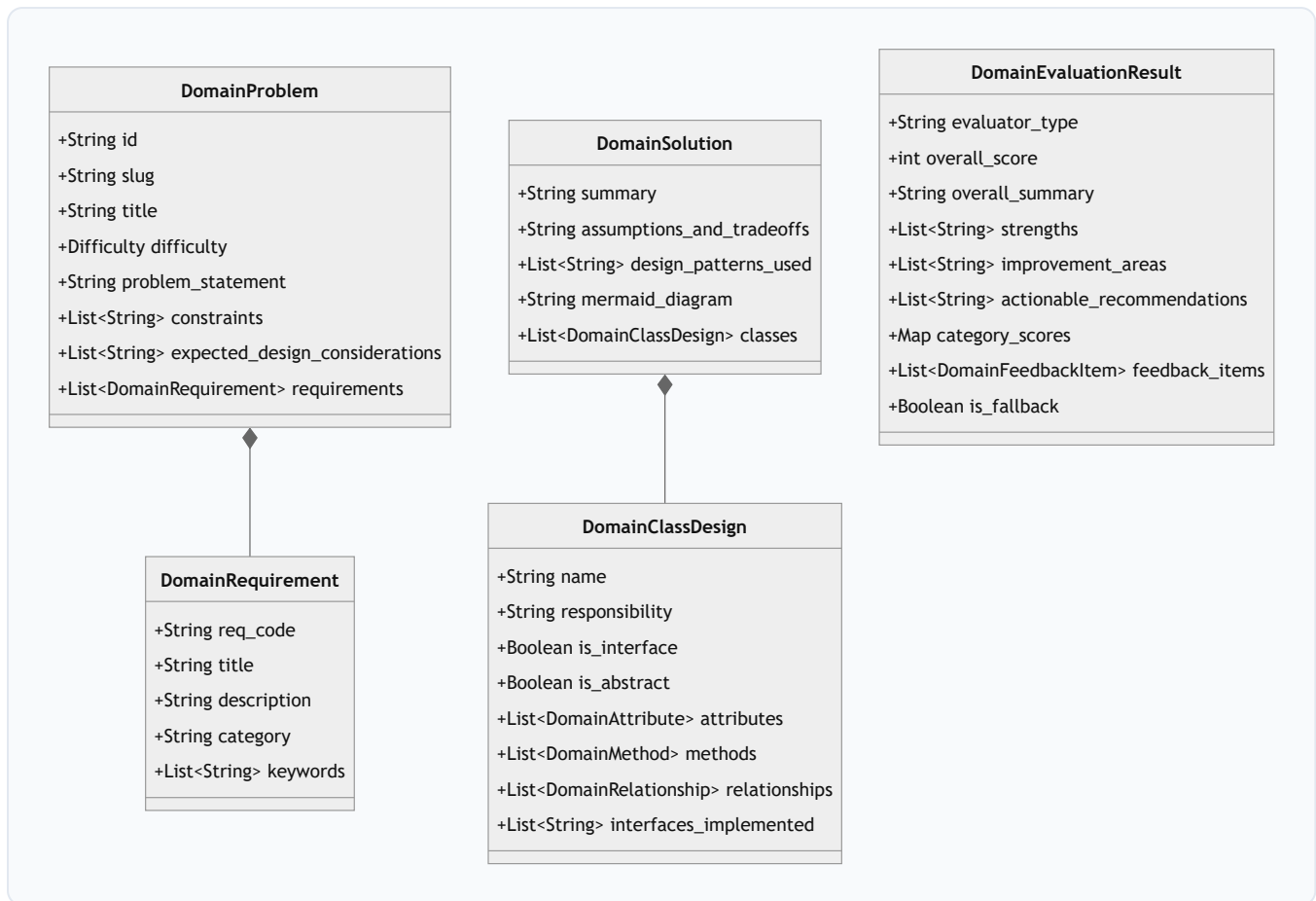


Syntax error in text
mermaid version 10.9.8

3. Domain Model Design

The core domain is structured around clean data classes and interfaces located in `backend/core/domain/`, decoupled from framework-specific ORM concerns.

3.1 Domain Entities



4. Evaluator Architecture (Strategy & Composite Pattern)

To ensure the platform never hardcodes vendor lock-in or breaks when AI credentials are absent, evaluation is built around the **Dependency Inversion Principle (DIP)**:

```

class SolutionEvaluator(ABC):
    @abstractmethod
    def evaluate(self, solution: DomainSolution, problem: DomainProblem) → DomainEvaluationResult:
        pass
  
```

4.1 Evaluation Implementations

1. **RuleBasedEvaluator** (Deterministic Engine):

- Evaluates 8 deterministic passes:
 - *Requirements Keyword & Semantic Mapping* (checks entity/method coverage against stated requirements).
 - *Single Responsibility Principle & God Class Detection* (identifies classes handling excessive methods or orthogonal domains).
 - *Abstraction & Interface Utilization* (verifies polymorphic contracts and interface implementation).
 - *SOLID Principles Compliance* (checks OCP, LSP, ISP, and DIP indicators).
 - *Coupling & Cohesion Analysis* (detects dangling class references where targets are missing).

- *Design Pattern Verification* (validates claimed patterns against structural classes).
 - *Edge Cases & Concurrency Notes* (scans for race condition/locking considerations).
 - *Overall Architecture Explanation Clarity*.
2. **AIEvaluator (Qualitative Reasoning):**
 - Provider-agnostic bridge supporting Google Gemini (`gemini-1.5-flash`) and OpenAI (`gpt-4o-mini`).
 - Sends structured prompt containing candidate classes, methods, relationships, diagram, and requirements.
 - Enforces strict JSON schema output mapping directly to `DomainEvaluationResult` .
 3. **CompositeEvaluator (Hybrid Orchestrator):**
 - Executes `RuleBasedEvaluator` for baseline ground truth.
 - Executes `AIEvaluator` inside a guarded try-except block.
 - Merges results (40% deterministic + 60% AI reasoning), deduplicates strengths and recommendations, and tags every finding with `DETERMINISTIC` or `AI_SUGGESTION` .
 - **Graceful Fallback:** If AI fails, times out, or has no API key, it seamlessly returns the deterministic result with `is_fallback=True` , ensuring learner work is never lost.

5. Attempt Lifecycle & Immutability

Practice attempts follow a strict state machine:



Syntax error in text
mermaid version 10.9.8

Immutability Invariant

Once an attempt transitions out of `DRAFT` , any subsequent `PUT` or `PATCH` request to modify classes or solution text is rejected with HTTP 400 (`"Cannot modify attempt in SUBMITTED/COMPLETED state. Submitted attempts are immutable."`).

To practice again, the learner uses the **Try Again** action, which instantiates a new `PracticeAttempt` with `attempt_number = N + 1` for the same problem. This preserves historical learning progression.

6. Key Architectural Trade-offs

Decision	Chosen Approach	Alternative Considered	Engineering Rationale
Architecture Topology	Modular Monolith	Microservices (Auth, Evaluator, Problems)	A focused 2-day MVP prioritized domain correctness, explainable feedback, and fast local development over distributed microservice complexity.
Evaluator Concurrency	Synchronous service execution with fast LLM timeouts (25s)	Celery / Redis asynchronous worker queue	Avoids heavy Redis/worker dependencies for local evaluation while keeping the service layer decoupled so workers can be introduced later with zero view changes.
Class Diagram Representation	Mermaid JS markdown rendering + auto-sync from structured classes	Drag-and-drop canvas (e.g. React Flow / Fabric.js)	Structured classes guarantee semantic data for static evaluation, while Mermaid provides immediate, code-like visualization without high canvas complexity.
Data Normalization	Relational <code>LLDProblem</code> , <code>PracticeAttempt</code> , <code>Solution</code> , <code>ClassDesign</code> , <code>Evaluation</code> , and <code>FeedbackItem</code>	Single JSON Document in MongoDB	Relational integrity ensures strong querying, cascading deletion of attempts, and precise feedback item categorization.